Design and Implementation of a Face Biometric Module: Anti-Spoofing and Open-Set Recognition

Colasuonno Giovanni — 2046000
Biometric Systems, Sapienza University of Rome

June 27, 2026

Abstract :

I design and implement a complete face biometric module that couples *presentation attack detection* (face anti-spoofing) with face *recognition*. Two anti-spoofing approaches are compared on the large-scale CelebA-Spoof dataset — a classic color-texture baseline (LBP features with an SVM) and a fine-tuned CNN — and combined through score-level fusion. I further assess generalisation in a cross-dataset setting and add an open-set recognition stage, obtaining a full anti-spoof → identity pipeline with a real-time demo. The report is organised by implementation steps, from the experimental setup to the complete biometric system.

1 Experimental Setup

I implemented the whole module in **Python**, relying on the OpenCV/PyTorch ecosystem. I split the work into two complementary tracks — a *hand-crafted* pipeline and a *deep-learning* pipeline — so that the same task can be tackled with classic and modern tools and the two can be compared fairly. In the hand-crafted pipeline I design the features myself (texture descriptors) and a classic classifier learns on top of them; in the deep-learning pipeline the network instead learns the features directly from the raw pixels during training.

Software. The main libraries and their role are summarised in Table 1. PyTorch (with the `timm` model zoo) is used for the convolutional network; `scikit-image` and `scikit-learn` provide the texture features and the SVM of the classic baseline; OpenCV handles image I/O, face detection and the live demo; `facenet-pytorch` supplies the face-embedding network used by the recognition stage.

Hardware. Training and inference run on a single **NVIDIA RTX 2060 SUPER** GPU (8 GB VRAM) with CUDA. GPU acceleration is what makes CNN fine-tuning practical.

Table 1: Tools used and their role in the module.

Component	Tool / library
Deep model (CNN)	PyTorch + `timm` (ResNet-18)
Classic baseline	`scikit-image` (LBP) + `scikit-learn` (SVM)
Face recognition	`facenet-pytorch` (MTCNN + InceptionResnetV1)
Image I/O & demo	OpenCV
Metrics & plots	NumPy, `scikit-learn`, Matplotlib

Reproducibility. All dependencies are installed in an isolated virtual environment and pinned in a `requirements.txt` file. Random operations that affect the experiments — most importantly the training/validation split — use a fixed seed, so that results can be reproduced exactly.

2 Dataset and Protocol

Dataset. I train and evaluate the anti-spoofing models on **CelebA-Spoof** [2], one of the largest public face anti-spoofing datasets. I use the *cropped* version (faces already tightly cropped) so that the models focus on the face rather than on the background. The choice of a large and public dataset is deliberate: a convolutional network needs tens of thousands of examples to learn without overfitting, and using a standard benchmark makes the results comparable with the literature. Each image carries a binary label: *live* (a genuine face, label 0) or *spoof* (a presentation attack such as a print, a replay on a screen, or a paper mask, label 1).

Protocol. I follow the dataset's official split and add an internal validation set, obtaining the three classic subsets of a biometric experiment (Table 2):

- **train** — the data the models learn from;
- **validation** — a 15% slice of the training set, used only to pick the operating *threshold* and to stop the CNN training early (it is *not* used for learning);
- **test** — the official, held-out split, used once for the final evaluation.

Table 2: Dataset split (CelebA-Spoof, cropped).

Subset	Images	Live	Spoof
Train (fit)	67,306	33.0%	67.0%
Validation	11,877	34.1%	65.9%
Test (disjoint)	66,787	29.7%	70.3%

Avoiding data leakage. The cropped version does not expose the *subject identity*, so I cannot build a per-subject split by myself. This matters for honesty of the evaluation. I therefore keep the validation set, carved at random from the training data, only for early stopping and threshold selection: it may share subjects with the training set, but this is harmless because it never contributes to learning. The **test set comes from the official, subject-disjoint split**, so the final numbers are measured on people the models have never seen — not just on unseen images.

Class imbalance. The data is skewed roughly 2:1 towards spoof. Left untreated, a model could reach a deceptively low error by simply predicting "spoof" most of the time. I counter this in the two pipelines with balanced sampling (classic baseline) and class weights (CNN), described in the corresponding sections. About 955 corrupted rows (empty image field) were detected and discarded during extraction.

3 Preprocessing and Augmentation

Since the faces are already cropped, preprocessing for the deep pipeline reduces to resizing each image to 224×224 and normalising it with the **ImageNet** mean and standard deviation. The normalisation is not arbitrary: the ResNet-18 backbone is pre-trained on ImageNet and therefore expects inputs on that colour scale, which makes fine-tuning more stable.

Augmentation. At training time I apply a chain of random transformations to every image (Fig. 1): random resized crop, horizontal flip, colour jitter, Gaussian blur, *random JPEG re-compression* and random erasing. Augmentation has two purposes. First, it prevents the network from memorising irrelevant details (overfitting), because it never sees exactly the same image twice. Second, and more specific to anti-spoofing, it improves **robustness**: presentation attacks alter texture, illumination and compression, so blur and especially random JPEG imitate the artefacts of print and replay attacks. This is the kind of variability that matters in the cross-dataset setting discussed later. At evaluation time no augmentation is applied — only the deterministic resize and normalisation — so that the test conditions are fixed.

The classic baseline uses a different, much lighter preprocessing (a smaller resize and conversion to chrominance colour spaces) that is part of its feature extraction and is described together with the baseline.

Figure 1. Training augmentation: several random variants of the same face.

4 Classic Baseline: Color-Texture Features + SVM

The first track is the *hand-crafted* one: I design a feature vector by hand and feed it to a classic classifier. It serves as a reference — a sensible error rate that the deep model must beat. The method follows the color-texture approach of Boulkenafet *et al.* [3], built on a simple observation: a genuine face and a *photo of a face* share the same shape but differ in their micro-texture — a screen adds pixel/moiré patterns, a print adds grain and gloss, re-compression adds artefacts. These cues are captured well by **Local Binary Patterns (LBP)** and are more visible in the colour (chrominance) channels than in plain luminance.

The feature vector. For each image I extract LBP histograms and concatenate them into a single descriptor, built along three axes:

- **Colour spaces:** I convert the image to **YCbCr** and **HSV** (two spaces that separate luminance from chrominance) and process all three channels of each.
- **Multi-scale LBP:** I compute the uniform LBP at two scales — $(P{=}8, R{=}1)$, capturing fine texture (10 bins), and $(P{=}16, R{=}2)$, capturing coarser texture (18 bins).
- **Histograms:** each (colour space, channel, scale) combination gives one normalised histogram; concatenating them yields the descriptor.

The dimensionality is therefore 2 spaces$\times$3 channels$\times$(10+18) bins = **168** features per image (Table 3). Images are resized to 128×128 before extraction, and the histograms are density-normalised so that the descriptor does not depend on image size.

Classifier. On top of these descriptors I train a **Support Vector Machine** with an RBF kernel ($C{=}10$, `gamma=scale`). The RBF kernel lets the SVM draw curved decision

Table 3: Composition of the 168-dimensional color-texture descriptor.

LBP scale	Bins	Histograms (2×3)
(P=8, R=1)	10	$6 \rightarrow 60$
(P=16, R=2)	18	$6 \rightarrow 108$
Total		**168**

boundaries, which is needed because live and spoof are not linearly separable in this feature space. Two choices handle the practical issues: the features are standardised (zero mean, unit variance, with statistics estimated on the training set only, to avoid leakage), and the SVM is fit on a *balanced* subsample of 12,000 images (6,000 per class) — both because the RBF-SVM does not scale to tens of thousands of samples and to neutralise the 2:1 class imbalance. The model is still evaluated on the full test set.

Score. Rather than a hard live/spoof decision, I take the signed distance from the decision boundary (`decision_function`) as a continuous "spoofness" score. A continuous score is what later allows moving the decision threshold and drawing the evaluation curves.

5 Baseline Results

Table 4 reports the baseline on the full test set. The color-texture SVM reaches an **EER of 7.69%** with an AUC of **0.976** — a strong result for a purely hand-crafted method, well inside the 10–25% range typically expected from texture baselines, and the reference the deep model has to beat. Figure 2 shows the same operating point from the threshold's perspective: as the threshold grows the SFAR (attacks accepted) rises while the FFR (live rejected) falls, and the EER is exactly where the two curves cross.

Operating point. At a threshold fixed on the validation set the ACER on test is 8.4%, and at a fixed APCER of 5% the baseline still rejects 12.8% of genuine faces (BPCER) [10] — the price of a hand-crafted detector at a strict security setting.

Variability. A single train/test split gives no sense of variance, so as a robustness check [1] I run a **5-fold cross-validation** of the SVM on a balanced in-distribution subsample, stratified by class: the EER is 1.18% ± 0.12% across folds, so the model is not at the mercy of one particular partition. Two caveats keep this honest. First, the cropped data has no identity labels, so the folds cannot be made subject-disjoint: the same subjects may fall in different folds, which makes this 1.18% an *optimistic*, in-distribution figure, not comparable to the 7.69% measured on the subject-disjoint test. Second, the ±0.12% is the spread of *this* experiment, not a confidence interval around the test EER. The large distance between the two therefore mixes two effects — the harder subject-disjoint test and the subject overlap inside the CV — and should not be read as pure domain difficulty.

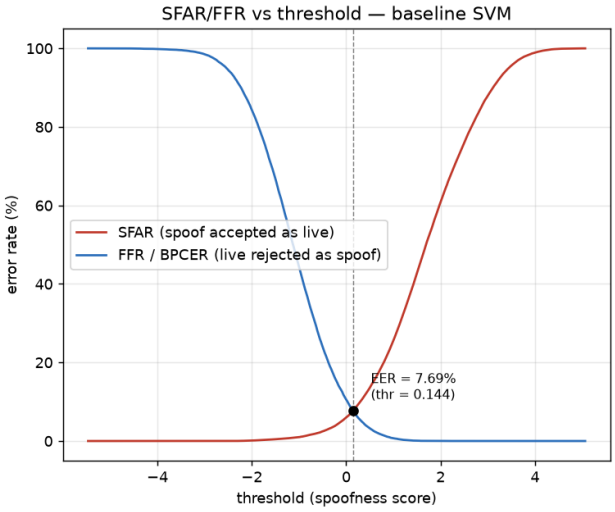

Figure 2. SFAR (spoof accepted as live) and FFR/BPCER (live rejected as spoof) of the baseline as a function of the decision threshold; the EER is the crossing point.

Table 4: Baseline (color-LBP + SVM) on the test set (66,787 images). Operating-point metrics use a threshold fixed on validation.

Metric	Value
EER	7.69%
AUC	0.976
5-fold CV EER (in-distrib.)	1.18 ± 0.12%
BPCER @ APCER=1% / 5%	44.2% / 12.8%
ACER @ validation threshold	8.43%

6 Deep Model: Fine-Tuned CNN

The second track is the *deep-learning* one. Instead of designing the features by hand, I let a convolutional network learn them from the raw pixels. I start from a **ResNet-18** [4] **pre-trained on ImageNet**, replace its final layer with a two-output head (live/spoof) and *fine-tune* the whole network on CelebA-Spoof. Transfer learning is the key idea: the backbone already knows how to see edges, textures and shapes, so I only teach it the new live/spoof distinction — which gives strong results with little data and little time.

Training. The network is trained for a few epochs with the AdamW optimiser (learning rate 3×10^{-4}, cosine schedule) and a batch size of 64. Three ingredients address the practical issues:

- **Mixed precision (AMP)** — computing in 16-bit where possible keeps the model within the 8 GB of VRAM and speeds training up.
- **Class weights** — the loss weights the rarer (live) class more, so the 2:1 imbalance does not push the network towards always predicting "spoof".
- **Early stopping** — after each epoch I measure the EER on the validation set and keep the best checkpoint, stopping when it no longer improves; this avoids overfitting.

Training takes about 24 minutes on the GPU (~5 epochs). The training curve (Fig. 3) shows the loss decreasing while the validation EER drops to roughly 0.5%, with both curves going down together — a sign that the model is not overfitting. The score used for evaluation is the softmax probability of the spoof class.

Results. On the full test set the CNN reaches an **EER of 3.69%** with an AUC of **0.994** (Table 5). It therefore *more than halves* the error of the hand-crafted baseline (7.69% → 3.69%) and is markedly closer to a perfect detector. This confirms the expectation: the features learned by the network beat the ones I engineered by hand. The SFAR/FFR curves (Fig. 4) cross at a much lower error than the baseline. Note that the CNN is very confident — its scores concentrate near 0 and 1 — so the crossing point sits close to a threshold of 0. At a fixed APCER of 5% the CNN rejects only 2.6% of genuine faces (BPCER), against 12.8% for the SVM: a lower EER *and* a more usable operating curve.

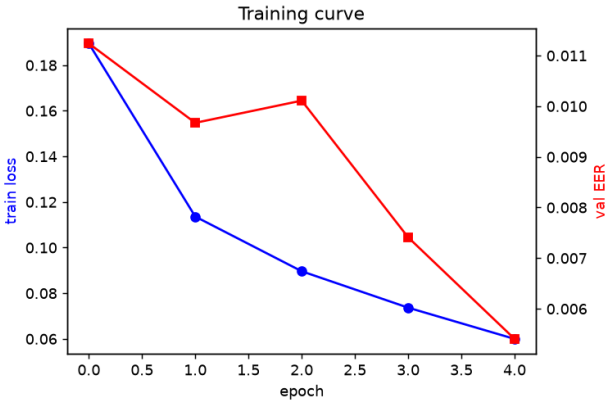

Figure 3. Training curve: training loss (left axis) and validation EER (right axis) per epoch.

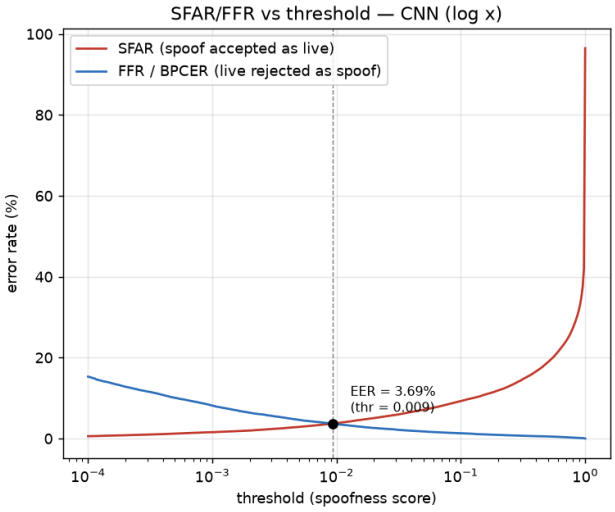

Figure 4. SFAR (spoof accepted) and FFR/BPCER (live rejected) of the CNN versus the decision threshold; the crossing point is the EER (3.69%).

Table 5: Fine-tuned CNN (ResNet-18) on the test set. Operating-point metrics use a threshold fixed on validation.

Metric	Value
EER	3.69%
AUC	0.994
BPCER @ APCER=1% / 5%	11.5% / 2.6%
ACER @ validation threshold	7.91%

7 Evaluation: Baseline vs CNN

So far each model was measured on its own; here I compare them head-to-head. Both are evaluated with the same code, the same metrics and the same subject-disjoint test set, so the comparison is fair.

Table 6 puts the two models side by side and the overlaid ROC curves are shown in Fig. 5. The CNN improves every metric: it *more than halves* the EER (7.69% → 3.69%) and raises the AUC from 0.976 to 0.994. The ROC makes the gap visual — the CNN curve lies above the baseline over the whole range, i.e. it is better at *every* operating threshold, not just at the EER. In short, on this dataset the learned features clearly outperform the hand-crafted ones, and the CNN is the model I carry forward to the cross-dataset, fusion and

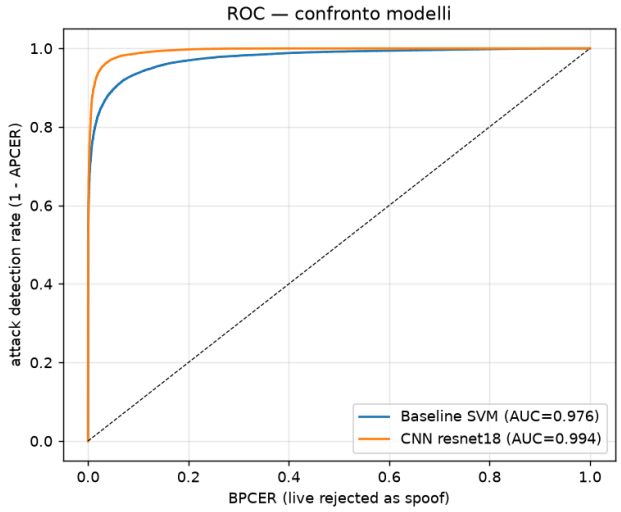

Figure 5. ROC curves of the baseline and the CNN on the test set; the CNN dominates at every operating point.

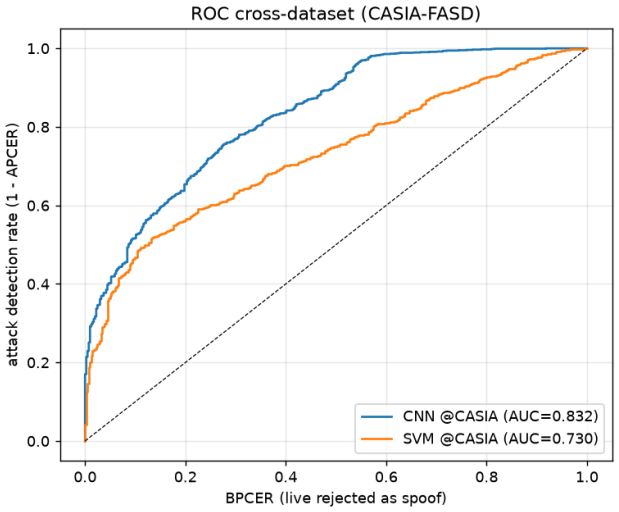

Figure 6. ROC curves on the unseen CASIA-FASD dataset. Both models lose a lot of discriminative power, but the CNN (AUC 0.832) still generalises better than the hand-crafted baseline (AUC 0.730).

recognition stages.

Table 6: Baseline vs CNN on the test set (66,787 images).

Metric	SVM (LBP)	CNN
EER	7.69%	**3.69%**
AUC	0.976	**0.994**
BPCER@APCER=5%	12.8%	**2.6%**

8 Cross-Dataset Evaluation: the Domain Gap

All the numbers above were measured on the CelebA-Spoof test set: a different *split*, but the same dataset, the same cameras and the same kinds of attack seen in training. A spoof detector that only works on the data it was trained on is of little practical use, so the real question is how it behaves on a domain it has *never* seen. This is the cross-dataset (or cross-domain) setting, and it is where presentation-attack detectors are known to struggle.

Protocol. I evaluate both models on a second, independent dataset — **CASIA-FASD** [5] (cropped), 1,655 faces — without any re-training or re-tuning on it. Crucially, the decision threshold is the one fixed on the *source* validation set (CelebA-Spoof) and is applied unchanged to the new data; tuning a threshold on the target would be unrealistic, since in deployment one does not have labelled attacks from the target domain. I report EER and AUC (which are threshold-free) together with the HTER — here HTER = (SFAR + FFR)/2, the half total error of the *counter-measure* at that fixed threshold, which is the same quantity as the ACER reported earlier — and it is precisely the metric that exposes how well the chosen operating point transfers.

Results. Table 7 contrasts the intra-dataset and cross-dataset performance, and Fig. 6 shows the cross-dataset ROC curves. The drop is large for both models: the CNN HTER goes from 7.9% to 35.4% (+27.5 points) and the SVM from 8.4% to 44.5% (+36.0 points). It is important to read this correctly: it is mostly a *threshold-transfer* failure, not a collapse of discriminative power. The CNN still keeps a clearly useful AUC of 0.832 on the unseen domain (far from the 0.5 of a random detector); what breaks is that the operating point chosen on the source no longer sits in the right place for the target's score distribution — the classic *domain gap*.

Table 7: Intra- vs cross-dataset performance. Threshold fixed on the source (CelebA-Spoof) validation set.

Model	Scenario	EER	AUC	HTER
CNN	intra (CelebA)	3.69%	0.994	7.91%
CNN	**cross (CASIA)**	**26.51%**	**0.832**	**35.36%**
SVM	intra (CelebA)	7.69%	0.976	8.43%
SVM	**cross (CASIA)**	**34.43%**	**0.730**	**44.45%**

Discussion. Two things are worth noting. First, the ranking between the models is preserved: even when both degrade, the CNN keeps a clear edge over the baseline (AUC 0.832 vs 0.730), so the learned features generalise *better* than the hand-crafted ones — they do not simply memorise the source domain. Second, the gap between the threshold-free EER (26.5%) and the fixed-threshold HTER (35.4%) for the CNN tells me that part of the problem is *calibration*: the model still separates live from spoof reasonably well on CASIA, but the threshold learned on CelebA sits in the wrong place for the new score distribution. This is an honest lim-

itation of the system, and it is exactly what motivates the lightweight on-site calibration step in the demo, where a handful of frames from the target setup are used to re-place the threshold.

9 Score-Level Fusion

The two detectors look at the face in different ways — hand-crafted texture versus learned features — so they may fail on different images. This is the setting of *multibiometric* systems: combining several matchers to do better than the best single one. I combine them at the **score level**, i.e. I merge the two spoofness scores without retraining anything.

Score normalisation. The scores are not directly comparable: the SVM `decision_function` is unbounded, while the CNN outputs a softmax probability in $[0, 1]$. Summing them as they are would let the SVM dominate just because of its larger range, so they must first be normalised onto a common scale, with every parameter estimated on the validation set only. **Min-max** normalisation assumes the minimum and maximum the matcher can produce are known — which is exactly false for an unbounded SVM score: the bounds can only be estimated on validation, a single outlier shifts them, and at test time scores can fall outside the range. I therefore do not trust min-max blindly but compare it against two alternatives better suited to this case: **z-score** (the most common) and **median/MAD** (robust to outliers) [1]. Table 8 reports the sum and the tuned weighted rule under each.

Table 8: Effect of the normalisation on the fused score (test EER%). The robust median/MAD is the only one under which the tuned weighted rule does not overfit.

Normalisation	sum	weighted
min-max	**3.35%**	5.14%
z-score	3.82%	5.15%
median/MAD	3.65%	**3.22%**

Fusion rules. On the normalised scores I try the classic combination rules: sum, mean, product, max, min and a weighted sum (weight tuned on the validation EER). The product, max and min rules only make sense on scores bounded in $[0, 1]$, so the full rule sweep (Table 9) is shown under min-max; sum and the weighted rule, which are well defined for any normalisation, are the ones compared across all three in Table 8. The corresponding ROC is in Fig. 7.

Discussion. The best simple fusion improves the EER from 3.69% to **3.35%**, and the product rule gives the highest AUC (0.9948). The gain is modest, and the reason is informative: fusion helps most when the two matchers are of comparable strength, whereas here the CNN already dominates the baseline, leaving little to add. The normalisation comparison (Table 8) is more revealing. Under min-max and z-score the *weighted* sum — whose weight is tuned on validation — *worsens* the test EER to ∼5.1%: the tuned weight overfits a fragile normalisation. Under the robust median/MAD the same tuned weighted rule instead gives the best result of all (**3.22%**). In other words, the weighted fusion was not intrinsically bad; it was sitting on a normalisation (min-max on an unbounded score) too brittle to carry a tuned parameter. The robust normalisation is the more defensible choice here.

Table 9: Fusion rules under min-max normalisation (test set).

Method	EER	AUC
SVM (only)	7.69%	0.976
CNN (only)	3.69%	0.994
Fusion sum / mean	**3.35%**	0.991
Fusion product	3.50%	**0.9948**
Fusion weighted (w=0.10)	5.14%	0.987

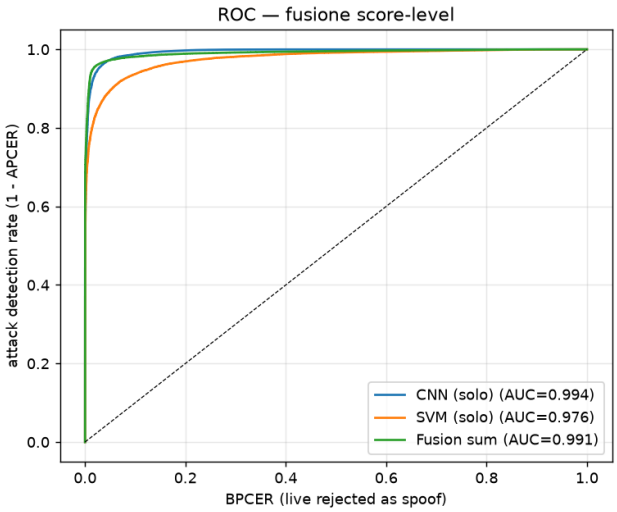

Figure 7. ROC of the best score-level fusion compared with the two single models.

10 From Liveness to Identity: Face Recognition

Up to here the system only answers *"is this a live face or an attack?"*. That is a presentation-attack detector, not yet a biometric *recognition* system: it never asks *"who is this person?"*. To turn the project into a complete face-biometric module I add an identification stage on top of the anti-spoofing one, so that the two run in a cascade: a frame is first checked for liveness, and only if it passes is it matched against the enrolled identities.

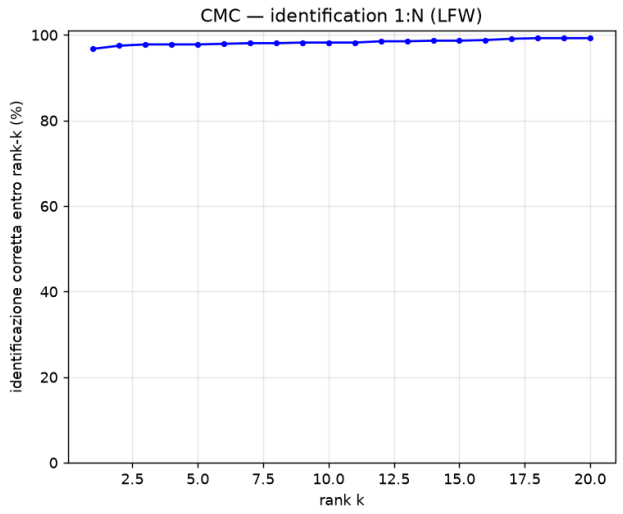

Figure 8. Cumulative Match Characteristic on LFW: the probe's correct identity is the top match 96.8% of the time, and within the top five 97.8% of the time.

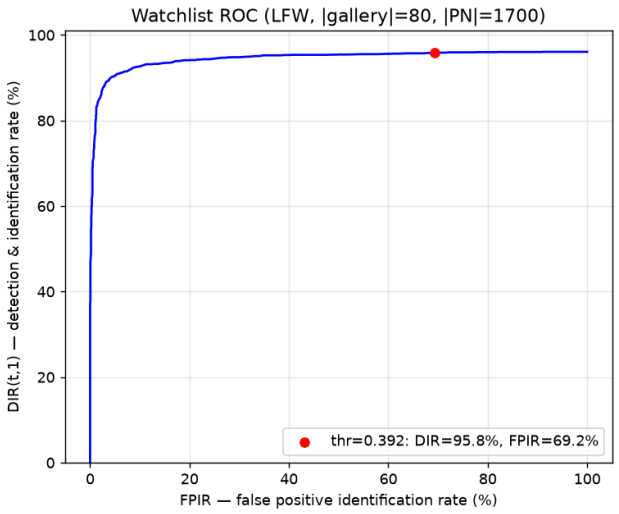

Figure 9. Watchlist ROC on LFW: detection-and-identification rate vs false positive identification rate. The red dot is the verification-calibrated threshold, which sits at a 69% FPIR.

Why a separate, pre-trained model. Recognition is a different problem from anti-spoofing and needs different features — not "is this texture real?" but "whose face is this?". Moreover, the anti-spoofing dataset (CelebA-Spoof, cropped) carries no identity labels, so it cannot be used to train a recognizer at all. I therefore rely on transfer learning from a model already trained on millions of faces: face detection and alignment with **MTCNN** [6], and a **FaceNet** [7] embedder (InceptionResnetV1 pre-trained on VGGFace2 [8]) that maps an aligned face to a 512-dimensional *embedding*. Two faces are then compared by the **cosine similarity** of their embeddings — high for the same person, low for different people. No training on my side; the recognition quality comes entirely from the pre-trained representation.

Benchmark on LFW. Since my anti-spoof data has no identities, I measure the recognizer on the standard **LFW** [9] benchmark, in the two canonical biometric tasks. In *verification* (1:1, "are these two faces the same person?") I reach an EER of **3.30%** (AUC 0.974), which fixes the cosine acceptance threshold at 0.392. In *identification* (1:N, "which gallery identity does this probe belong to?") I obtain a rank-1 accuracy of **96.77%** and rank-5 of 97.80%; the full CMC curve is shown in Fig. 8. These numbers confirm the embedding is strong enough to build an identification stage on.

Open-set / watchlist evaluation. The realistic scenario is *open-set*: most people presenting to the system are *not* enrolled, so it must be allowed to answer "unknown" instead of being forced to pick the closest gallery name. Evaluating this needs the proper watchlist metrics [1]: the **DIR**$(t, 1)$ (detection-and-identification rate — a genuine probe is both ranked first *and* above threshold), the FNIR $= 1 - \mathrm{DIR}(t, 1)$, and the **FPIR** (the watchlist false-alarm rate, an impostor whose best match exceeds t). I therefore build a meaningful protocol on LFW: 80 watchlist identities (one enrolment photo each), giving 2,544 genuine probes, against 1,700 impostor probes from 78 *disjoint* identities. The Watchlist ROC (DIR vs FPIR) is in Fig. 9 and the operating points in Table 10.

The key — and humbling — finding is that the threshold calibrated on 1:1 verification (0.392) does *not* transfer to the watchlist: at that value the DIR is high (95.8%) but the FPIR is a useless **69%**: most impostors raise a false alarm. The operating point must instead be read off the watchlist ROC: at a 1% FPIR the system still identifies 76.5% of enrolled subjects, and at 10% FPIR, 92.7%. This is the same lesson as Section 8 — a threshold chosen in one scenario is the wrong one in another — now made quantitative for identification.

Table 10: Open-set watchlist on LFW (80 enrolled, 2,544 genuine and 1,700 impostor probes). The verification threshold is far too permissive for this task.

Operating point	DIR(t,1)	FNIR	FPIR
thr = verif. (0.392)	95.8%	4.2%	69.2%
FPIR = 10%	92.7%	7.3%	10%
FPIR = 1%	76.5%	23.5%	1%

A qualitative demo on my own data. I also wired the full anti-spoof → identity cascade and tried it on a tiny personal gallery (two enrolled subjects) with my own genuine photos and impostors. With only two enrolled identities this is a *qualitative demo, not a measurement* — the gallery

is far too small for the rates above to mean anything, and the genuine probes risk sharing the acquisition session with the enrolment shot, which inflates the apparent performance. With that caveat, the system does behave as intended: genuine subjects are identified and impostors are rejected. One detail is instructive: several impostor phone photos were stopped one step *earlier*, by the anti-spoofing gate, because heavy messaging-app JPEG compression looks to the CNN like a replay attack — the very same domain gap of Section 6, surfacing again as an over-rejection. It confirms that the liveness stage, not the recognition stage, is the current bottleneck of the full system.

11 Conclusions

I built a complete face-biometric module step by step, from data to a real-time demo, and at each step I compared design choices instead of taking them for granted. On CelebA-Spoof the fine-tuned CNN clearly beat the classic color-texture baseline, more than *halving* the EER (7.69% → 3.69%), and a score-level fusion of the two pushed it a little further, to 3.22% with a robust median/MAD normalisation and a weighted rule. On top of the anti-spoofing stage I added a pre-trained FaceNet recogniser (96.8% rank-1 on LFW) and evaluated it as a proper open-set watchlist (DIR/FNIR/FPIR), which exposed that the verification threshold is far too permissive for identification — the operating point has to be chosen on the watchlist ROC.

The honest result of the project, however, is generalisation. The same CNN that looks almost perfect on CelebA loses most of its *calibration* on an unseen domain (HTER 7.9% → 35.4% on CASIA-FASD, while the AUC stays at 0.83), and the very same threshold-transfer problem reappeared in the watchlist evaluation and in the demo's compression-induced false alarms. Measuring these honestly — rather than reporting only the favourable intra-dataset numbers — is, to me, the most important outcome.

The natural next steps follow directly from this gap: stronger domain-generalisation training (heavier augmentation, mixing several spoof datasets) and the lightweight on-site calibration already sketched in the demo, which replaces the decision threshold from a handful of frames captured on the target device. Closing that gap, rather than squeezing the last decimal of intra-dataset EER, is where a deployable system would actually be won.

References

[1] M. De Marsico *et al. Biometric Systems — course material and lecture notes (BIPlab).* Sapienza University of Rome, A.Y. 2025/26.

[2] Y. Zhang, Z. Yin, Y. Li, G. Yin, J. Yan, J. Shao, Z. Liu. *CelebA-Spoof: Large-Scale Face Anti-Spoofing Dataset with Rich Annotations.* ECCV, 2020.

[3] Z. Boulkenafet, J. Komulainen, A. Hadid. *Face Spoofing Detection Using Colour Texture Analysis.* IEEE Trans. Information Forensics and Security, 2016.

[4] K. He, X. Zhang, S. Ren, J. Sun. *Deep Residual Learning for Image Recognition.* CVPR, 2016.

[5] Z. Zhang, J. Yan, S. Liu, Z. Lei, D. Yi, S. Z. Li. *A Face Antispoofing Database with Diverse Attacks.* ICB, 2012.

[6] K. Zhang, Z. Zhang, Z. Li, Y. Qiao. *Joint Face Detection and Alignment Using Multi-task Cascaded Convolutional Networks.* IEEE Signal Processing Letters, 2016.

[7] F. Schroff, D. Kalenichenko, J. Philbin. *FaceNet: A Unified Embedding for Face Recognition and Clustering.* CVPR, 2015.

[8] Q. Cao, L. Shen, W. Xie, O. M. Parkhi, A. Zisserman. *VGGFace2: A Dataset for Recognising Faces across Pose and Age.* IEEE FG, 2018.

[9] G. B. Huang, M. Ramesh, T. Berg, E. Learned-Miller. *Labeled Faces in the Wild: A Database for Studying Face Recognition in Unconstrained Environments.* Univ. Massachusetts, TR 07-49, 2007.

[10] ISO/IEC 30107-3. *Information Technology — Biometric Presentation Attack Detection — Part 3: Testing and Reporting.* 2017.